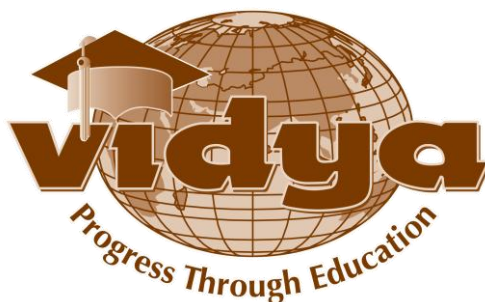


Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSL 203 OBJECT ORIENTED PROGRAMMING LAB

LAB MANUAL

PREPARED BY:

Ms. ATHULYA BHASKAR K R

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

PROGRAM EDUCATIONAL OBJECTIVES (PEOS)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

PROGRAM SPECIFIC OUTCOMES (PSOS)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

PROGRAM OUTCOMES (POS)

Computer Science & Engineering graduates will be able to:

PO-1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO-2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO-4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO-6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO-8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO-10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CSL 203	OBJECT ORIENTED PROGRAMMING LAB (IN JAVA)	CATEGOR Y	L	T	P	CREDIT	YEAR OF INTRODUCTION
		PCC	0	0	3	2	2019

Preamble: The aim of the course is to provide hands-on experience to the learners on various object oriented concepts in Java Programming. This course helps the learners to enhance the capability to design and implement various Java applications for real world problems.

Prerequisite: Topics covered under the course Programming in C (EST 102)

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Use network related commands and configuration files in Linux Operating System.
CO2	Develop network application programs and protocols.
CO3	Analyze network traffic using network monitoring tools.
CO4	Design and set up a network and configure different network protocols.
CO5	Develop simulation of fundamental network concepts using a network simulator.

CO Number	PO STRENGTH												PSO STRENGTH	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO1	PSO2
CO1	3	3	3					2		2		2	2	
CO2	3	3	3	2				2		2		2	2	2
CO3	3	3	3	2	2			2		2		2	2	2
CO4	3	3	3	2	2	2		2		2		2	2	
CO5	3	3	3	2	3			2		2		2	2	

Abstract POs defined by National Board of Accreditation

PO#	Broad PO	PO#	Broad PO
-----	----------	-----	----------

PO1	Engineering Knowledge	PO7	Environment and Sustainability
PO2	Problem Analysis	PO8	Ethics
PO3	Design/Development of solutions	PO9	Individual and team work
PO4	Conduct investigations of complex problems	PO10	Communication
PO5	Modern tool usage	PO11	Project Management and Finance
PO6	The Engineer and Society	PO12	Life long learning

Assessment Pattern

Bloom's Category	Continuous Assessment Test - Internal Exam (Percentage)	End Semester Examination (Percentage)
Remember	20	20
Understand	20	20
Apply	60	60
Analyse		
Evaluate		
Create		

Mark Distribution

Total Marks	CIE Marks	ESE Marks	ESE Duration
150	75	75	3 hours

Continuous Internal Evaluation Pattern:

Attendance : 15 marks

Continuous Evaluation in Lab : 30 marks

Continuous Assessment Test : 15 marks

Viva-voce : 15 marks

Internal Examination Pattern: The marks will be distributed as Algorithm 30 marks, Program 20 marks, Output 20 marks and Viva 30 marks. Total 100 marks which will be converted out of 15 while calculating Internal Evaluation marks. End Semester Examination Pattern: The marks will be distributed as Algorithm 30marks, Program 20 marks, Output 20 marks and Viva 30 marks. Total 100 marks will be converted out of 75 for End Semester Examination

Operating System to Use in Lab : Linux

Compiler/Software to Use in Lab : gcc, javac, jdk, jre, Eclipse, NetBeans,MySQL /

PostgreSQL.

Programming Language to Use in Lab : Java

Fair Lab Record:

All Students attending the Object Oriented Programming Lab (in Java) should have a Fair Record. The fair record should be produced in the University Lab Examination. Every experiment conducted in the lab should be noted in the fair record. For every experiment in the fair record the right hand page should contain Experiment Heading, Experiment Number, Date of Experiment, Aim of Experiment, Operations Performed, Details of Experiment including algorithm and Result of Experiment. The left hand page should contain a print out of the code used for the experiment and sample output obtained for a set of input.

SYLLABUS

The syllabus contains six sessions (A, B, C, D, E, F). Each session consists of three concrete Java exercises, out of which at least two questions are mandatory.

(A) Basic programs using datatypes, operators, and control statements in Java.

1) Write a Java program that checks whether a given string is a palindrome or not.

Ex: MALAYALAM is palindrome.

2) Write a Java Program to find the frequency of a given character in a string. **

3) Write a Java program to multiply two given matrices. **

(B) Object Oriented Programming Concepts: Problem on the use of constructors, inheritance, method overloading & overriding, polymorphism and garbage collection:

4) Write a Java program which creates a class named 'Employee' having the following members: Name, Age, Phone number, Address, Salary. It also has a method named 'printSalary()' which prints the salary of the Employee. Two classes 'Officer' and 'Manager' inherits the 'Employee'

class. The 'Officer' and 'Manager' classes have data members 'specialization' and 'department' respectively. Now, assign name, age, phone number, address and salary to an officer and a manager by making an object of both of these classes and print the same. (Exercise to understand inheritance).

5) Write a java program to create an abstract class named Shape that contains an empty method named numberOfSides(). Provide three classes named Rectangle, Triangle and Hexagon such that each one of the classes extends the class Shape. Each one of the classes contains only the method numberOfSides() that shows the number of sides in the given geometrical structures. (Exercise to understand polymorphism).

6) Write a Java program to demonstrate the use of garbage collector.

(C) Handling different types of files as well as input and output management methods:

7) Write a file handling program in Java with reader/writer.

8) Write a Java program that read from a file and write to file by handling all file related exceptions. **

9) Write a Java program that reads a line of integers, and then displays each integer, and the sum of all the integers (Use String Tokenizer class of java.util). **

(D) Exception handling and multi-threading applications:

11) Write a Java program that shows the usage of try, catch, throws and finally.**

12) Write a Java program that implements a multi-threaded program which has three threads. First thread generates a random integer every 1 second. If the value is even, second thread computes the square of the number and prints. If the value is odd the third thread will print the value of cube of the number.

13) Write a Java program that shows thread synchronization. **

(E) Graphics Programming:

14) Write a Java program that works as a simple calculator. Arrange Buttons for digits and the + - * % operations properly. Add a text field to display the result. Handle any possible exceptions like divide by zero. Use Java Swing.**

15) Write a Java program that simulates a traffic light. The program lets the user select one of three lights: red, yellow, or green. When a radio button is selected, the light is turned on, and only one light can be on at a time. No light is on when the program starts. **

16) Write a Java program to display all records from a table using Java Database Connectivity (JDBC).

(F) Standard Searching and Sorting Algorithms using data structures and algorithms learned from course Data Structures (CST 201):

Write a Java program for the following: **

1. Create a doubly linked list of elements.
2. Delete a given element from the above list.
3. Display the contents of the list after deletion.
4. Write a Java program that implements Quick sort algorithm for sorting a list of names in ascending order. **

17) Write a Java program that implements Quick sort algorithm for sorting a list of names in ascending order. **

18) Write a Java program that implements the binary search algorithm.

GENERAL INSTRUCTIONS (for the lab)**Preparation:**

- Read the theory, algorithm, and expected output for the experiment scheduled before the lab session. Preparation is key to successful implementation.
- Familiarize yourself with the basic functionality of the Linux/Unix environment you will be using (e.g., shell navigation, basic command utilities).
- Bring your Lab Manual/Record to every session to document procedures, code snippets, and results in real-time.
- For simulation exercises (e.g., Scheduling, Deadlock), prepare the required input values (process details, resource matrix, reference strings) beforehand.

Lab Setup & Tools:

- Ensure you are logged into the assigned Linux/Unix machine or virtual environment.
- Familiarize yourself with the command-line editor (e.g., nano, vim) or IDE (e.g., VS Code, Eclipse) available for C development.
- Confirm the availability and basic usage of the GNU Compiler Collection (gcc) for compiling your C/C++ programs.
- Create a new directory for each experiment (e.g., mkdir exp_04_fork) to keep your source code and executables organized.

Coding Standards & Implementation:

- Language: All system programming tasks (IPC, Synchronization, Process Management) must be implemented using C or C++, utilizing standard system calls.
- Header Comments: Start every C/C++ file with a comment block including your name, roll number, date, experiment number, and the program's purpose.
- System Calls: Use robust error handling (checking the return value against -1 and using perror()) immediately after every critical system call (e.g., fork(), exec(), shmget(), semop()).
- Compilation Flags: Use necessary flags, such as -o to name the executable and -pthread when compiling multithreaded programs.

Example: `gcc myprogram.c -o myprog -pthread`

Code Clarity: Use clear, descriptive variable names for PIDs, file descriptors, semaphores, and shared memory keys.

Testing & Debugging:

- Test Frequency: Compile and test your code in small, incremental steps, especially after implementing a new system call.
- Tracing: Utilize diagnostic commands (`ps`, `top`, `pstree`) to verify process creation and state changes. Use `strace` to trace the system calls made by your program, which is crucial for debugging.
- Synchronization Check: For concurrency exercises (Semaphores, Mutexes), test your program multiple times to ensure that race conditions are eliminated and synchronization works consistently.
- Resource Cleanup: For IPC programs (Message Queues, Shared Memory), ensure you explicitly clean up allocated resources after the program finishes using appropriate commands (`ipcrm` or system calls).

Documentation & Submission:

- Demonstration: Your code must be demonstrated live to the instructor or TA. Be prepared to explain the logic and the role of key system calls.
- Viva Voce: Be ready to answer questions regarding the underlying OS concepts (e.g., difference between `fork` and `exec`, semaphore operations, scheduling logic).
- Lab Record: All components (Aim, Theory, Algorithm, Code, Procedure, Observed Result) must be formally entered into your record book.
- Submission: Submit the requested source files (`.c` or `.cpp`) and the output/result sheet according to the instructor's specifications (e.g., file naming convention: `UCSEST105_ExptX_RollNo.c`).

Safety & Conduct:

- System Safety: NEVER attempt to run system calls or commands that could compromise the stability or security of the operating system (e.g., using `kill -9` on essential system processes).
- System Login: Always log off your account before leaving the lab.
- Ask for Help: If you encounter persistent compilation errors, segmentation faults, or unexpected behavior from a system call, ask the instructor immediately.

INDEX

Exp No.	Experiment Name	Page Number
1	PALINDROME OR NOT	12
2	FREQUENCY OF CALCULATOR	15
3	MATRIX MULTIPLICATION	18
4	INHERITANCE	22
5	ABSTRACT CLASS	28
6	GARBAGE COLLECTION	31
7	FILE HANDLING & EXCEPTION HANDLING	33
8	STRING TOKENIZER	35
9	EXCEPTION HANDLING USING TRY, CATCH, THROW, THROWS AND FINALLY	38
10	MULTITHREADS	41
11	THREAD SYNCHRONIZATION	44
12	SIMPLE CALCULATOR USING SWING	47
13	TRAFFIC LIGHT SIMULATION USING SWING	56
14	DOUBLY LINKED LIST	61
15	QUICK SORT	64
16	BINARY SEARCH	68

EXP NO: 1**PALINDROME OR NOT****AIM**

To develop a Java program that checks whether a given string is a palindrome by comparing its characters from both ends.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand the concept of palindrome strings.
- Apply string manipulation techniques in Java.
- Use conditional statements and loops for logical comparison.
- Implement Java String methods such as `length()` and `charAt()`.
- Develop problem-solving skills using basic object-oriented programming concepts.

DESCRIPTION

A palindrome is a sequence of characters that reads the same in both forward and reverse directions. Examples of palindromes include **MADAM**, **MALAYALAM**, and **LEVEL**.

In Java, a palindrome can be checked by comparing the first character with the last character, the second character with the second-last character, and so on until the middle of the string is reached. If all corresponding characters are identical, the string is declared a palindrome; otherwise, it is not.

This experiment demonstrates the use of Java String methods, looping constructs, conditional statements, and user input through the Scanner class.

ALGORITHM

1. Start the program.
2. Import the required Java packages.
3. Read the input string from the user.
4. Determine the length of the string.
5. Initialize a flag variable to indicate whether the string is a palindrome.
6. Compare the first character with the last character, the second character with the second-last character, and continue until the middle of the string.
7. If any pair of characters does not match, set the flag and terminate the loop.
8. If the flag remains unchanged, display "**Palindrome**".
9. Otherwise, display "**Not Palindrome**".
10. Stop the program.

PROGRAM

```
import java.util.Scanner;
class Test {
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the String: ");
        String str = sc.nextLine();

        int flag = 0;
        int len = str.length();

        for (int i = 0; i < len / 2; i++) {

            if (str.charAt(i) != str.charAt(len - i - 1)) {
                flag = 1;
                break;
            }
        }

        if (flag == 0)
            System.out.println("Palindrome");
        else
            System.out.println("Not Palindrome");

        sc.close();
    }
}
```

EXPECTED OUTPUT**Sample Output 1**

Enter the String: MALAYALAM

Palindrome

Sample Output 2

Enter the String: COMPUTER

Not Palindrome

MODEL VIVA QUESTIONS

1. What is a palindrome?
2. Which Java class is used to read user input?
3. What is the purpose of the charAt() method?
4. What is the function of the length() method?
5. What is the difference between `==` and `.equals()` when comparing strings in Java?

RESULT

The Java program was successfully implemented and executed. The program correctly determines whether the given input string is a palindrome by comparing characters from both ends and displays the appropriate result.

EXP NO: 2**FREQUENCY OF A CHARACTER****AIM**

To develop a Java program that determines the frequency (number of occurrences) of a specified character in a given string.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand the concept of string processing in Java.
- Read input from the user using the Scanner class.
- Traverse a string using loops.
- Utilize the charAt() and length() methods of the String class.
- Implement conditional statements to count character occurrences.

DESCRIPTION

A string is a sequence of characters enclosed within double quotation marks. In many text-processing applications, it is often necessary to determine how many times a particular character appears in a string.

This program accepts a string and a character from the user. It then scans the string character by character using a loop. Whenever the specified character matches the current character in the string, a counter is incremented. After traversing the entire string, the program displays the total frequency of the given character.

This experiment demonstrates the use of Java strings, loops, conditional statements, user input handling, and basic counting algorithms.

ALGORITHM

1. Start the program
2. Import the required Java packages.
3. Read a string from the user.
4. Read the character whose frequency is to be determined.

5. Initialize a counter variable to zero.
6. Traverse the string from the first character to the last character.
7. Compare each character of the string with the given character.
8. If both characters are equal, increment the counter.
9. Repeat Steps 6–8 until the end of the string is reached.
10. Display the frequency of the specified character.
11. Stop the program.

PROGRAM

```
import java.util.Scanner;

class Test {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the String: ");
        String str = sc.nextLine();

        System.out.print("Enter the Character: ");
        char ch = sc.nextLine().charAt(0);

        int count = 0;

        for (int i = 0; i < str.length(); i++) {

            if (str.charAt(i) == ch) {
                count++;
            }
        }

        System.out.println("Frequency of '" + ch + "' = " + count);

        sc.close();
    }
}
```

EXPECTED OUTPUT**Sample Output**

Enter the String: java programming

Enter the Character: a

Frequency of 'a' = 3

Sample Output

Enter the String: Object Oriented Programming

Enter the Character: g

Frequency of 'g' = 2

MODEL VIVA QUESTIONS

1. What is a string in Java?
2. Which package contains the Scanner class?
3. What is the purpose of the charAt() method?
4. What does the length() method return?
5. Why is a loop required in this program?

RESULT

The Java program was successfully implemented and executed. The program correctly counts and displays the number of occurrences of a specified character in the given string using string traversal and conditional statements.

EXP NO: 3**MATRIX MULTIPLICATION****AIM**

To develop a Java program to perform multiplication of two matrices and display the resultant matrix.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand the concept of matrix multiplication.
- Represent matrices using two-dimensional arrays in Java.
- Apply nested loops for matrix operations.
- Perform arithmetic operations using arrays.
- Develop logical programming skills for multidimensional data.

DESCRIPTION

A matrix is a rectangular arrangement of numbers organized in rows and columns. Matrix multiplication is a fundamental operation in mathematics and computer science, widely used in graphics, machine learning, scientific computing, and data analysis. For matrix multiplication to be valid, the number of columns in the first matrix must be equal to the number of rows in the second matrix. Each element of the resultant matrix is obtained by multiplying the corresponding row elements of the first matrix with the corresponding column elements of the second matrix and summing the products. This experiment demonstrates the use of two-dimensional arrays, nested loops, arithmetic operations, and user input in Java.

ALGORITHM

1. Start the program.
2. Read the order (rows and columns) of the first matrix.
3. Read the order (rows and columns) of the second matrix.

4. Check whether the number of columns of the first matrix equals the number of rows of the second matrix.
5. If the condition is not satisfied, display an appropriate error message and terminate the program.
6. Read the elements of both matrices.
7. Multiply the matrices using three nested loops.
8. Store the result in a third matrix.
9. Display the resultant matrix.
10. Stop the program.

PROGRAM

```
import java.util.Scanner; class MMatrix{
public static void main(String args[])
{ Scanner sc = new Scanner(System.in); System.out.print("Enter the order – m1:");
int m1 = sc.nextInt(); System.out.print("Enter the order – n1:");
    }
    int A[][] = new int[m1][n1];
    int B[][] = new int[m2][n2];
    int C[][] = new int[m1][n2];
    System.out.println("Enter Elements of mat1 ");
    for(int i=0;i<m1;i++)
    {
        for(int j=0;j<n1;j++)
        { System.out.print("A["+i+"]"+"["+j+"]=");
            A[i][j] = sc.nextInt();
        }
    }
    System.out.println("Enter Elements of mat2 ");
    for(int i=0;i<m2;i++)
    {
        for(int j=0;j<n1;j++)
        { System.out.print("B["+i+"]"+"["+j+"]=");
            B[i][j] = sc.nextInt();
        }
    }
}
```

```
        for(int i=0;i<m1;i++){ for(int
            j=0;j<n2;j++){
                C[i][j]=0;
                for(int k=0;k<n1;k++){
                    C[i][j] += A[i][k] * B[k][j];
                }
            }
        }
        System.out.println("Product");
        for(int i=0;i<m1;i++){
for(int j=0;j<n2;j++)
{ System.out.print(C[i][j]+"\\t");
}

                System.out.println();
        }
    }
}
```

EXPECTED OUTPUT

Sample Output

Enter rows and columns of Matrix A

2 2

Enter rows and columns of Matrix B

2 2

Enter elements of Matrix A

1 2

3 4

Enter elements of Matrix B

5 6

7 8

Resultant Matrix

19 22

43 50

MODEL VIVA QUESTIONS

1. What is a matrix?

2. What is a two-dimensional array?
3. What is the condition for matrix multiplication?
4. Why are three nested loops required?
5. What is the purpose of the third matrix?

RESULT

The Java program was successfully implemented and executed. The program correctly performs matrix multiplication using two-dimensional arrays and displays the resultant matrix.

EXP NO: 4**INHERITANCE****AIM**

To develop a Java program that demonstrates the concept of inheritance by creating a derived class from a base class and accessing inherited members.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand the concept of inheritance in object-oriented programming.
- Create base and derived classes in Java.
- Reuse data members and methods through inheritance.
- Implement code reusability using the **extends** keyword.
- Differentiate between parent and child classes.

DESCRIPTION

Inheritance is one of the fundamental principles of Object-Oriented Programming (OOP). It enables a new class (derived class or subclass) to inherit the properties and behaviors of an existing class (base class or superclass). This promotes code reusability, modularity, and maintainability.

In Java, inheritance is implemented using the **extends** keyword. The derived class can access the public and protected members of the base class and may also introduce new members or override inherited methods to provide specialized behavior.

This experiment demonstrates how inheritance allows the creation of hierarchical class relationships while reducing code duplication.

ALGORITHM

1. Start the program.
2. Define a base class with data members and member functions.
3. Create a derived class using the **extends** keyword.
4. Add additional members or methods to the derived class if required.

5. Create an object of the derived class.
6. Access the inherited members and methods.
7. Display the required output.
8. Stop the program.

PROGRAM

```
import java.util.Scanner; class Employee{
    private String name;
    private int age;
    private long phoneNumber;
    private String address; private
    double salary;

    public void setName(String name){
        this.name = name;
    }
    public void setAge(int age){ this.age=age;
    }
    public void setPhoneNumber(long phoneNumber)
    { this.phoneNumber = phoneNumber;
    }
    public void setAddress(String address){ this.address = address;
    }
    public void setSalary(double salary)
    { this.salary = salary;
    }
    public double printSalary()
    {
        return salary;
    }
    public String getName()
    {
        return name;
    }
    public int getAge()
```

```
        {
            return age;
        }
public String getAddress()
{
    return address;
}
public long getPhoneNumber(){ return
    phoneNumber;
}
}

class Officer extends Employee{ private
    String specialization;
    private String department;

    public void setSpecialization(String specialization){ this.specialization =
        specialization;
    }
    public void setDepartment(String department){ this.department = department;
    }

    public String getDepartment()
    {

        return department;
    }
    public String getSpecialization()
    {
        return specialization;
    }
}
Class Manager extends Employee
{
```

```
private String specialization; private String department;

public void setSpecialization(String specialization)
{
    this.specialization = specialization;
}

public void setDepartment(String department)
{ this.department = department;
}

public String getDepartment()
{ return department;
}

public String getSpecialization()
{ return specialization;
}

}

class Test{
public static void main(String args[])
{ Scanner sc = new Scanner(System.in); Officer o = new
    Officer();
    System.out.println("Enter the officer's Detail");
    System.out.print("Name:"); o.setName(sc.nextLine());
    System.out.print("Address:"); o.setAddress(sc.nextLine());
    System.out.print("Specialization:");
    o.setSpecialization(sc.nextLine());
    System.out.print("Department:");
    o.setDepartment(sc.nextLine()); System.out.print("Age:");
    o.setAge(sc.nextInt());
    System.out.print("Number:");
    o.setPhoneNumber(sc.nextLong());
    System.out.print("Salary:");
    o.setSalary(sc.nextDouble()); sc.nextLine(); // to
```

```
        skip new Line System.out.println("The officer
        Detail");
        System.out.println("Name:"+o.getName());
        System.out.println("Age:"+o.getAge());
        System.out.println("Number:"+o.getPhoneNumber());
        System.out.println("Address:"+o.getPhoneNuber());
        System.out.println("Salary:"+o.printSalary());
        System.out.println("Specialization:"+o.getSpecialization());
        System.out.println("Department:"+o.getDepartment());
        Manager m = new Manager(); System.out.println("Enter the
        manager's Detail"); System.out.print("Name:");
        m.setName(sc.nextLine()); System.out.print("Address:");
        m.setAddress(sc.nextLine());
        System.out.print("Specialization:");
        m.setSpecialization(sc.nextLine());
        System.out.print("Department:");
        m.setDepartment(sc.nextLine()); System.out.print("Age:");
        m.setAge(sc.nextInt());
        System.out.print("Number:");
        m.setPhoneNumber(sc.nextLong());
        System.out.print("Salary:");
        m.setSalary(sc.nextDouble()); sc.nextLine(); // to
        skip new Line System.out.println("The manager's
        Detail");
        System.out.println("Name:"+m.getName());
        System.out.println("Age:"+m.getAge());
        System.out.println("Number:"+m.getPhoneNumber());
        System.out.println("Address:"+m.getPhoneNuber());
        System.out.println("Salary:"+m.printSalary());
        System.out.println("Specialization:"+m.getSpecialization());
        System.out.println("Department:"+m.getDepartment());
    }
}
```

EXPECTED OUTPUT**Sample Output**

Employee Details

Employee ID : 101

Employee Name : Rahul

Department : Computer Science

Salary : 45000

(Modify the sample output to match your actual Java program if it uses different classes or data.)

MODEL VIVA QUESTIONS

1. What is inheritance?
2. What are the advantages of inheritance?
3. What is a superclass?
4. What is a subclass?
5. Which keyword is used to implement inheritance in Java?

RESULT

The Java program was successfully implemented and executed. The concept of inheritance was demonstrated by creating a derived class that inherits the properties and methods of a base class, thereby illustrating code reusability and hierarchical class relationships.

EXP NO: 5**ABSTRACT CLASS****AIM**

To develop a Java program that demonstrates the implementation of an abstract class and abstract methods.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

1. Understand the concept of abstraction in Object-Oriented Programming.
2. Define and implement abstract classes and abstract methods.
3. Differentiate between abstract classes and concrete classes.
4. Apply inheritance to implement abstract methods.
5. Develop reusable and maintainable Java programs.

DESCRIPTION

Abstraction is one of the important features of Object-Oriented Programming that hides implementation details and exposes only the essential functionality to the user. In Java, abstraction is achieved using **abstract classes** and **interfaces**.

An abstract class cannot be instantiated directly. It may contain both abstract methods (without implementation) and concrete methods (with implementation). Any class that extends an abstract class must provide implementations for all its abstract methods.

This experiment demonstrates how abstract classes improve code reusability and provide a common structure for derived classes.

ALGORITHM

1. Start the program.
2. Define an abstract class containing one or more abstract methods.
3. Create a subclass using the **extends** keyword.
4. Implement all abstract methods in the subclass.
5. Create an object of the subclass.
6. Invoke the implemented methods.
7. Display the output.
8. Stop the program.

PROGRAM

```
abstract class Shape
{
    public abstract void numberOfSides();
}

class Rectangle extends Shape
{
    public void numberOfSides()
    {
        System.out.println("Number of Sides of Rectangle = 4");
    }
}

class Triangle extends Shape
{
    public void numberOfSides()
    {
        System.out.println("Number of Sides Triangle= 3");
    }
}

class Hexagon extends Shape
{
    public void numberOfSides()
    {
        System.out.println("Number of Sides Hexagon = 6");
    }
}

class Test
{
    public static void main(String args[])
    {
        Rectangle r = new Rectangle();
        Triangle t = new Triangle();
        Hexagon h = new Hexagon();
        r.numberOfSides();
        t.numberOfSides();
        h.numberOfSides();
    }
}
```

EXPECTED OUTPUT

Area of Circle = 78.5

Area of Rectangle = 50.0

(Modify according to your program.)

MODEL VIVA QUESTIONS

1. What is an abstract class?
2. Can an abstract class be instantiated?
3. What is an abstract method?
4. Which keyword is used to declare an abstract class?
5. Can an abstract class contain constructors?
6. Can abstract classes contain concrete methods?
7. Difference between abstract class and interface.
8. Why is abstraction required?
9. Can a class contain both abstract and non-abstract methods?
10. Give one real-life example of abstraction.

RESULT

The Java program was successfully implemented and executed. The concept of abstraction was demonstrated using an abstract class and its derived class.

EXP NO: 6**GARBAGE COLLECTION****AIM**

To develop a Java program that demonstrates the working of the Garbage Collector.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Understand memory management in Java.
- Explain the role of the Garbage Collector.
- Use the `System.gc()` method.
- Understand object destruction.
- Improve memory utilization in Java applications.

DESCRIPTION

Java automatically manages memory using the Garbage Collector (GC). Objects that are no longer referenced become eligible for garbage collection. The Garbage Collector frees the memory occupied by such objects automatically.

Although the execution of the Garbage Collector cannot be forced, it can be requested using the `System.gc()` method.

This experiment demonstrates automatic memory management in Java.

ALGORITHM

1. Start.
2. Create objects.
3. Remove object references.
4. Request garbage collection using `System.gc()`.
5. Display the output.
6. Stop.

PROGRAM

```
class GCollector
{
    //Finalize() is called by garbage collector before the object is
    destroyed.
    protected void finalize()
    {
        System.out.println("Garbage Collector invoked finalize() method.");
    }
}
class DemoGCollector
{
    public static void main(String[] args)
    {
        GCollector obj=new GCollector();

        //obj no longer references to an object. Now it is available for Garbage
        collection. obj=null;

        //The garbage collector is invoked here.
        System.gc();
    }
}
```

EXPECTED OUTPUT

Object Created

Garbage Collector Called

Object Destroyed

MODEL VIVA QUESTIONS

1. What is Garbage Collection?
2. Why is Garbage Collection required?
3. What is System.gc()?
4. Can Garbage Collection be forced?
5. What is the purpose of the finalize() method?

RESULT

The program successfully demonstrated automatic memory management using the Java Garbage Collector.

EXP NO: 7**FILE HANDLING AND EXCEPTION HANDLING****AIM**

To develop a Java program that demonstrates file handling along with exception handling.

LEARNING OUTCOMES

Students will be able to:

- Create and access files in Java.
- Read and write files.
- Handle exceptions.
- Use try, catch, and finally.
- Improve program reliability.

DESCRIPTION

File handling enables Java programs to store and retrieve data from files. Since file operations may generate errors such as missing files or access denial, exception handling is essential.

Java provides `try`, `catch`, `finally`, `throw`, and `throws` mechanisms to handle runtime exceptions gracefully.

ALGORITHM

1. Start.
2. Create/open a file.
3. Perform read/write operations.
4. Enclose statements inside a try block.
5. Catch exceptions.
6. Display appropriate messages.
7. Close the file.
8. Stop.

PROGRAM

```
import java . io . * ;
public class FileHandling {
public static void main ( String args []) throws IOException
{
FileInputStream fi = null;
FileOutputStream ft= null;
try
{
FileInputStream fi = new FileInputStream ( "input.txt" );
FileOutputStream ft = new FileOutputStream ( "output.txt" ); int i = 0 ;
while (( i = fi . read () ) != - 1 )
{ ft . write ( i );
}
}
catch ( IOException e ) {
System . out . println ( "Exception " + e );
}}}
```

EXPECTED OUTPUT

File Created Successfully

Data Written Successfully

No Exception Occurred

MODEL VIVA QUESTIONS

1. What is exception handling?
2. Difference between checked and unchecked exceptions.
3. What is the purpose of finally?
4. Difference between throw and throws.
5. What is IOException?

RESULT

The program successfully demonstrated file handling and exception handling using Java.

EXP NO: 8**STRING TOKENIZER****AIM**

To develop a Java program using the StringTokenizer class to split a string into individual tokens.

LEARNING OUTCOMES

Students will be able to:

- Understand tokenization.
- Use the StringTokenizer class.
- Separate strings into tokens.
- Process user input efficiently.
- Apply string manipulation techniques.

DESCRIPTION

The StringTokenizer class in Java is used to divide a string into smaller parts called tokens based on delimiters such as spaces, commas, or special characters. It provides methods such as `hasMoreTokens()` and `nextToken()` to process each token sequentially.

ALGORITHM

1. Start.
2. Read a string.
3. Create a StringTokenizer object.
4. Use a loop while tokens are available.
5. Display each token.
6. Stop.

PROGRAM

```
import java.io.*; import java.util.*; class RunPoliDemo
{
    public static void main(String args[]) throws IOException
    {
```

```
        BufferedReader br=new BufferedReader(new
        InputStreamReader(System.in)); System.out.print("Enter a line of integers :
        ");
        String str = br.readLine();
        StringTokenizer st = new StringTokenizer(str);
        int sum=0;
        while (st.hasMoreTokens())
        {
            String token=st.nextToken();
            System.out.println("Integer : " + token);
            sum = sum + Integer.parseInt(token);
        }

        System.out.print("Sum of integers : "+sum);
    }
}
```

EXPECTED OUTPUT

Input String:

Java is Object Oriented

Output:

Java is Object Oriented

MODEL VIVA QUESTIONS

1. What is a token?
2. What is the purpose of StringTokenizer?
3. Which package contains StringTokenizer?
4. What is a delimiter?
5. Difference between split() and StringTokenizer.
6. What is hasMoreTokens()?
7. What is nextToken()?
8. Can multiple delimiters be used?
9. What is string parsing?

10. Applications of tokenization.

RESULT

The Java program successfully demonstrated the use of the StringTokenizer class to split a string into individual tokens.

EXP NO: 9**EXCEPTION HANDLING USING TRY, CATCH, THROW, THROWS AND FINALLY****AIM**

To develop a Java program demonstrating exception handling using **try**, **catch**, **throw**, **throws**, and **finally** blocks.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand runtime exceptions in Java.
- Handle exceptions using try and catch blocks.
- Use the throw keyword to generate exceptions.
- Declare exceptions using throws.
- Execute cleanup operations using finally.

DESCRIPTION

Exception handling is one of the important features of Java that enables a program to handle runtime errors without terminating abruptly. Java provides keywords such as **try**, **catch**, **throw**, **throws**, and **finally** to detect and manage exceptions efficiently.

The **try** block contains statements that may generate an exception. The **catch** block handles the exception. The **throw** keyword is used to explicitly generate an exception, while **throws** declares exceptions that may occur in a method. The **finally** block is always executed irrespective of whether an exception occurs or not.

ALGORITHM

1. Start the program.
2. Write statements that may generate an exception inside the try block.
3. Catch the generated exception using an appropriate catch block.
4. Use throw to generate a custom exception if required.
5. Declare exceptions using throws.
6. Execute the finally block.
7. Display the output.

8. Stop the program.

PROGRAM

```
import java.util.Scanner;

class AgeOverException extends Exception
{
    public AgeOverException(String str)
{
    System.out.println(str);
    }
}

public class ExceptionP
{
    public static void main(String[] args)
    {
        Scanner s = new Scanner(System.in); System.out.print("Enter your age :");
        int age = s.nextInt();
    try
        {
            If(age>60)
                throw new AgeOverException("Invalid age");
            else
                System.out.println("Valid age");
        }
    catch (AgeOverException a)
    {
        System.out.println(a);
    }
}
```

EXPECTED OUTPUT

Exception Caught Successfully

Finally Block Executed

MODEL VIVA QUESTIONS

1. What is an exception?
2. What is the difference between checked and unchecked exceptions?
3. What is the purpose of the try block?
4. Why is finally used?
5. Difference between throw and throws.

6. Can multiple catch blocks be used?

RESULT

The Java program was successfully executed and demonstrated exception handling using try, catch, throw, throws, and finally blocks.

EXP NO: 10**MULTITHREADING****AIM**

To develop a Java program that demonstrates multithreading using the Thread class and Runnable interface.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Understand the concept of multithreading.
- Create multiple threads.
- Execute concurrent tasks.
- Differentiate between Thread class and Runnable interface.
- Improve program efficiency using threads.

DESCRIPTION

Multithreading allows multiple parts of a program to execute simultaneously. Each thread represents an independent path of execution.

Java supports multithreading through the **Thread** class and the **Runnable** interface. Threads are useful in applications requiring background processing, animation, networking, and multitasking.

ALGORITHM

1. Start.
2. Create a class extending Thread or implementing Runnable.
3. Override the run() method.
4. Create thread objects.
5. Invoke start().

PROGRAM

```
import java.util.Random;

class X implements Runnable
{
```

```
static int random;

Random rand = new Random();

public void run() {
    random = rand.nextInt(25);
    System.out.println(random);
}

}

class Y implements Runnable
{
    public void run()
    {
        if (X.random % 2 == 0)
            System.out.println((int) Math.pow(X.random, 2) + "\n");
    }
}

class Z implements Runnable
{
    public void run()
    {
        if (X.random % 2 != 0)
            System.out.println((int) Math.pow(X.random, 3) + "\n");
    }
}

class main {
    public static void main(String args[])
    {
        for (int i = 0; i < 10; i++) {
            Thread objX = new Thread(new X());
            Thread objY = new Thread(new Y());
            Thread objZ = new Thread(new Z());
            try {
                Thread.sleep(1000);
            }
            catch (Exception e)
            {
            }
            objX.start();
            objY.start();
        }
    }
}
```

```
        objZ.start();  
    }  
}  
}
```

EXPECTED OUTPUT

Child Thread : Thread-0

Even Numbers

2
4
6
8
10

Main Thread

1
3
5
7
9

MODEL VIVA QUESTIONS

1. What is a thread?
2. Difference between process and thread.
3. What is multithreading?
4. How is a thread created?
5. Difference between start() and run().

RESULT

The Java program successfully demonstrated concurrent execution using multiple threads.

EXP NO: 11**THREAD SYNCHRONIZATION****AIM**

To develop a Java program demonstrating thread synchronization.

LEARNING OUTCOMES

Students will be able to:

1. Understand synchronization.
2. Prevent race conditions.
3. Share resources safely.
4. Use synchronized methods.
5. Improve thread safety.

DESCRIPTION

Synchronization ensures that only one thread accesses a shared resource at a time. Without synchronization, multiple threads may simultaneously modify shared data, resulting in inconsistent outputs.

Java provides the **synchronized** keyword to achieve mutual exclusion and maintain data integrity.

ALGORITHM

1. Start.
2. Create a shared resource.
3. Create multiple threads.
4. Synchronize access using synchronized.
5. Execute threads.
6. Display synchronized output.
7. Stop.

PROGRAM

```
class Main  
{
```

```
public static void main(String args[])
{
    Table obj = new Table();//only one object
    MyThread1 t1=new MyThread1(obj);
    MyThread2 t2=new MyThread2(obj);
    t1.start();
    t2.start();
}
}
class Table
{
    synchronized void printTable(int n){
    for(int i=1;i<=5;i++){
    System.out.println(n*i);
    Try
    {
        Thread.sleep(400);
    }
    catch(Exception e){System.out.println(e);}

    }
}
//end of the method
}
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table y)
    {
        t=y;
    }
    public void run()
    {
        t.printTable(5);
    }
}
```

```
class MyThread2 extends Thread{  
    Table t;  
    MyThread2(Table y){  
        t=y;  
    }  
    public void run(){  
        t.printTable(100);  
    }  
}
```

EXPECTED OUTPUT

Thread-1 accessing resource...

Completed

Thread-2 accessing resource...

Completed

MODEL VIVA QUESTIONS

1. What is synchronization?
2. Why is synchronization required?
3. What is race condition?
4. Which keyword is used?
5. What is mutual exclusion?

RESULT

The Java program successfully demonstrated synchronization of threads while accessing shared resources.

EXP NO: 12**SIMPLE CALCULATOR USING SWING****AIM**

To develop a Java Swing application that performs basic arithmetic operations such as addition, subtraction, multiplication, and division.

LEARNING OUTCOMES

Upon completion of this experiment, the student will be able to:

- Develop GUI applications using Java Swing.
- Use Swing components such as JButton, JTextField, JFrame, and JPanel.
- Handle button events using ActionListener.
- Perform arithmetic operations.
- Design interactive desktop applications.

DESCRIPTION

Swing is a part of the Java Foundation Classes (JFC) used to create platform-independent Graphical User Interface (GUI) applications.

In this experiment, a calculator is designed using Swing components. The application accepts two operands and performs arithmetic operations based on the selected operator. Event handling is implemented using the **ActionListener** interface.

ALGORITHM

1. Start the program.
2. Create a JFrame.
3. Add JTextField and JButton components.
4. Register ActionListener for each button.
5. Read the operands.
6. Perform the selected arithmetic operation.
7. Display the result.
8. Repeat until the application is closed.

9. Stop.

PROGRAM

```
import javax . swing . * ;
import java . awt . * ;
import java . awt . event . * ;
public class calculator
{
    calculator ()
    {
        JFrame jrm = new JFrame () ;
        jrm . setSize ( 500 , 500 ) ;
        JLabel jbl = new JLabel ( "Calculator" ) ;
        jbl . setBounds ( 227 , 10 , 400 , 30 ) ;
        jrm . add ( jbl ) ;
        JTextField jtf = new JTextField () ;
        jtf . setEditable ( false ) ;
        jtf . setBounds ( 50 , 50 , 400 , 30 ) ;
        jrm . add ( jtf ) ;
        JButton b1 , b2 , b3 , b4 , b5 , b6 , b7 , b8 , b9 , b0 , bdiv , bmul , bsub ,
        badd , bdec , beq , bdel , bclr , clear , dot ;
        b1 = new JButton ( "1" ) ;
        b2 = new JButton ( "2" ) ;
        b3 = new JButton ( "3" ) ;
        b4 = new JButton ( "4" ) ;
        b5 = new JButton ( "5" ) ;
        b6 = new JButton ( "6" ) ;
        b7 = new JButton ( "7" ) ;
        b8 = new JButton ( "8" ) ;
        b9 = new JButton ( "9" ) ;
        b0 = new JButton ( "0" ) ;
        bdiv = new JButton ( "/" ) ;
        bmul = new JButton ( "*" ) ;
        bsub = new JButton ( "-" ) ;
        badd = new JButton ( "+" ) ;
```

```
bdec = new JButton ( "." );
beq = new JButton ( "=" );
clear = new JButton ( "Clear" );
dot = new JButton ( "." );
b7 . setBounds ( 90 , 100 , 50 , 40 );
b8 . setBounds ( 180 , 100 , 50 , 40 );
b9 . setBounds ( 270 , 100 , 50 , 40 );
bdiv . setBounds ( 360 , 100 , 50 , 40 );
b4 . setBounds ( 90 , 170 , 50 , 40 );
b5 . setBounds ( 180 , 170 , 50 , 40 );
b6 . setBounds ( 270 , 170 , 50 , 40 );
bmul . setBounds ( 360 , 170 , 50 , 40 );
b1 . setBounds ( 90 , 240 , 50 , 40 );
b2 . setBounds ( 180 , 240 , 50 , 40 );
b3 . setBounds ( 270 , 240 , 50 , 40 );
bsub . setBounds ( 360 , 240 , 50 , 40 );
bdec . setBounds ( 90 , 310 , 50 , 40 );
b0 . setBounds ( 180 , 310 , 50 , 40 );
beq . setBounds ( 270 , 310 , 50 , 40 );
badd . setBounds ( 360 , 310 , 50 , 40 );
clear . setBounds ( 200 , 400 , 90 , 40 );
dot . setBounds ( 250 , 470 , 50 , 40 );jrm . add ( b7 );
jrm . add ( b8 );
jrm . add ( b9 );
jrm . add ( bdiv );
jrm . add ( b4 );
jrm . add ( b5 );
jrm . add ( b6 );
jrm . add ( bmul );
jrm . add ( b1 );
jrm . add ( b2 );
jrm . add ( b3 );
jrm . add ( bsub );
jrm . add ( bdec );
jrm . add ( b0 );
```

```
        jrm . add ( beq );
        jrm . add ( badd );
        jrm . add ( clear );
        jrm . add ( dot );
        jrm . setLayout ( null );
        jrm . setResizable ( false );
        jrm . setDefaultCloseOperation ( JFrame . EXIT_ON_CLOSE );
        jrm . setVisible ( true );
        b1 . addActionListener ( new ActionListener () {
            public void actionPerformed ((ActionEvent e) {
                if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
                    jtf . setText ( "" );
                    jtf . setText ( jtf . getText () + "1" );
            }
        });
        b2 . addActionListener ( new ActionListener ()
        {
            public void actionPerformed ((ActionEvent e)
            {
                if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
                    jtf . setText ( "" );
                    jtf . setText ( jtf . getText () + "2" );
            }
        });
        b3 . addActionListener ( new ActionListener ()
        {
            public void actionPerformed ((ActionEvent e)
            {
                if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
                    jtf . setText ( "" );
                    jtf . setText ( jtf . getText () + "3" );
            }
        });
    });
```

```
b4 . addActionListener ( new ActionListener () {  
    public void actionPerformed ((ActionEvent e )  
    {  
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not  
defined!" ))  
            jtf . setText ( "" );  
            jtf . setText ( jtf . getText () + "4" );  
    }  
} );  
b5 . addActionListener ( new ActionListener () {  
    public void actionPerformed ((ActionEvent e ) {  
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not  
defined!" ))  
            jtf . setText ( "" );  
            jtf . setText ( jtf . getText () + "5" );  
    }  
} );  
b6 . addActionListener ( new ActionListener ()  
{  
    public void actionPerformed ((ActionEvent e )  
    {  
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not  
defined!" ))  
            jtf . setText ( "" );  
            jtf . setText ( jtf . getText () + "6" );  
    }  
});  
b7 . addActionListener ( new ActionListener ()  
{  
    public void actionPerformed ((ActionEvent e )  
    {  
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not  
defined!" ))  
            jtf . setText ( "" );
```

```
jtf . setText ( jtf . getText () + "7" );
}
});
b8 . addActionListener ( new ActionListener ()
{
    public void actionPerformed ( ActionEvent e )
    {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
        jtf . setText ( jtf . getText () + "8" );
    }
} );
b9 . addActionListener ( new ActionListener () {
    public void actionPerformed ( ActionEvent e )
    {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
        jtf . setText ( jtf . getText () + "9" );
    }
} );
b0 . addActionListener ( new ActionListener () {
    public void actionPerformed ( ActionEvent e )
    {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
        jtf . setText ( jtf . getText () + "0" );
    }
} );
clear . addActionListener ( new ActionListener ()
{
    nPerformed ( ActionEvent e ) {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
```

```
jtf . setText ( "" );
jtf . setText ( "" );
}
} );
dot . addActionListener ( new ActionListener () {
    public void actionPerformed ((ActionEvent e) {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
            jtf . setText ( jtf . getText () + "." );
        }
    } );
badd . addActionListener ( new ActionListener () {
    public void actionPerformed ((ActionEvent e) {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
            jtf . setText ( jtf . getText () + "+" );
        }
    } );
bsub . addActionListener ( new ActionListener () {
    public void actionPerformed ((ActionEvent e) {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
            jtf . setText ( jtf . getText () + "-" );
        }
    } );
bmul . addActionListener ( new ActionListener () {
    public void actionPerformed ((ActionEvent e) {
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not
defined!" ))
            jtf . setText ( "" );
            jtf . setText ( jtf . getText () + "X" );
        }
    } );
```

```
});  
bdiv . addActionListener ( new ActionListener () {  
    public void actionPerformed ((ActionEvent e) {  
        if ( jtf . getText (). equals ( "Enter two operands!" ) || jtf . getText (). equals ( "Not  
defined!" ))  
            jtf . setText ( "" );  
            jtf . setText ( jtf . getText () + "/" );  
        }  
    });  
beq . addActionListener ( new ActionListener () {  
    public void actionPerformed ((ActionEvent e) {  
        try {  
            String S = jtf . getText ();  
            int i = 0 ;  
            char b = S . charAt ( i );  
            while ( b != '+' && b != '-' && b != 'X' && b != '/' && b != '%' )  
            {  
                i ++;  
                b = S . charAt ( i );  
            }  
            Float x , y , ans = 0f ;  
            x = Float . parseFloat ( S . substring ( 0 , i ));  
            y = Float . parseFloat ( S . substring ( i + 1 , S . length ()));  
            if ( b == '+' ) {ans = x + y ; }  
            else if ( b == '-' ) {  
                ans = x - y ;  
            } else if ( b == 'X' ) {  
                ans = x * y ;  
            } else if ( b == '/' ) {  
                ans = x / y ;  
            }  
            if ( b == '/' && y == 0 ) {  
                jtf . setText ( "Not defined!" );  
            } else {  
                jtf . setText ( ans + "" );  
            }  
        }  
    }  
});
```

```
        }  
        } catch ( Exception ex ) {  
            jtf . setText ( "Enter two operands!" );  
        }  
    }  
});  
}  
public static void main ( String [] args ) {  
    new calculator ();  
}  
}
```

EXPECTED OUTPUT

Operand 1 : 20

Operand 2 : 10

Operation : +

Result : 30

MODEL VIVA QUESTIONS

1. What is Swing?
2. Difference between AWT and Swing.
3. What is JFrame?
4. What is JButton?
5. What is ActionListener?

RESULT

The Java Swing application was successfully developed and executed. The program correctly performs basic arithmetic operations through a graphical user interface using Swing components and event handling.

EXP NO: 13**TRAFFIC LIGHT SIMULATION USING SWING****AIM**

To develop a Java Swing application that simulates the operation of a traffic light using radio buttons.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

Understand the concept of GUI programming using Java Swing.

Design graphical user interfaces using Swing components.

Use **JFrame**, **JPanel**, and **JRadioButton**.

Handle user events using the **ActionListener** interface.

Simulate real-world applications using event-driven programming.

DESCRIPTION

Java Swing provides a rich set of components for developing Graphical User Interface (GUI) applications. A traffic light simulator demonstrates event-driven programming where selecting a radio button changes the corresponding traffic light color.

Initially, all radio buttons are displayed with a white background. When the **Start** button is selected, it turns green; selecting **Wait** changes the corresponding button to yellow; selecting **Stop** changes it to red. At any time, only one light remains active, closely simulating the behavior of an actual traffic signal.

ALGORITHM

1. Start the program.
2. Create a **JFrame**.
3. Create panels and radio buttons labeled **Start**, **Wait**, and **Stop**.
4. Group the radio buttons using **ButtonGroup**.
5. Set the initial background color of all buttons to white.
6. Register an **ActionListener** for each radio button.
7. Change the selected button's background color to Green, Yellow, or Red accordingly.

8. Reset the remaining buttons to white.
9. Display the GUI.
10. Stop.

PROGRAM

```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;

public class traffic {

    traffic() {

        JFrame T = new JFrame("Traffic light");

        JPanel p1 = new JPanel();

        JPanel p2 = new JPanel();

        JPanel p3 = new JPanel();

        JPanel p4 = new JPanel();

        JPanel p5 = new JPanel();

        JPanel p6 = new JPanel();

        JPanel p7 = new JPanel();

        JRadioButton b[] = new JRadioButton[3];

        ButtonGroup bg = new ButtonGroup();

        b[0] = new JRadioButton("Start");

        b[0].setBackground(Color.WHITE);

        b[1] = new JRadioButton("WAIT");

        b[1].setBackground(Color.WHITE);

        b[2] = new JRadioButton("STOP");

        b[2].setBackground(Color.WHITE);

        bg.add(b[0]);

        bg.add(b[1]);

        bg.add(b[2]);
```

```
p1.setBackground(Color.BLACK);
p2.setBackground(Color.BLACK);
p3.setBackground(Color.BLACK);
p4.setBackground(Color.BLACK);
p5.setBackground(Color.BLACK);
p6.setBackground(Color.BLACK);
p7.setBackground(Color.BLACK);
p2.add(p4);
p2.add(b[0]);
p2.add(p5);
p2.add(b[1]);
p2.add(p6);
p2.add(b[2]);
p2.add(p7);
p2.setLayout(new GridLayout(1, 7));
T.add(p1);
T.add(p2);
T.add(p3);
T.setSize(450, 300);
T.setLayout(new GridLayout(3, 1));
T.setVisible(true);
T.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
b[0].addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        b[0].setBackground(Color.GREEN);
        b[1].setBackground(Color.WHITE);
        b[2].setBackground(Color.WHITE);
    }
}
```

```
});

b[1].addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        b[0].setBackground(Color.WHITE);

        b[1].setBackground(Color.YELLOW);

        b[2].setBackground(Color.WHITE);

    }

});

b[2].addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        b[0].setBackground(Color.WHITE);

        b[1].setBackground(Color.WHITE);

        b[2].setBackground(Color.RED);

    } });

}

public static void main(String[] args) {

    new traffic();

}

}
```

EXPECTED OUTPUT

Traffic Light Simulator

- Start
- Wait
- Stop

Selecting "Start" → Green

Selecting "Wait" → Yellow

Selecting "Stop" → Red

MODEL VIVA QUESTIONS

1. What is Swing?
2. What is a JRadioButton?
3. Why is ButtonGroup used?

4. What is event handling?
5. What is ActionListener?

RESULT

The Java Swing application was successfully developed and executed. The traffic light simulation correctly changes the selected signal using radio buttons and event handling.

EXP NO: 14**DOUBLY LINKED LIST****AIM**

To develop a Java program for creating a doubly linked list, deleting a specified element, and displaying the updated list.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

1. Understand the concept of a doubly linked list.
2. Create and manipulate linked list elements.
3. Delete nodes from a linked list.
4. Traverse linked lists using iterators.
5. Implement Java Collection Framework classes.

DESCRIPTION

A doubly linked list is a linear data structure in which each node contains data and two links: one pointing to the previous node and another to the next node. This structure allows traversal in both forward and backward directions.

Java provides the `LinkedList` class in the Collections Framework, which supports insertion, deletion, and traversal of elements efficiently. This experiment demonstrates creating a doubly linked list, removing elements, and displaying the updated list using iterators.

ALGORITHM

1. Start.
2. Import the `java.util` package.
3. Create a `LinkedList` object.
4. Insert elements into the list.
5. Display the list.
6. Remove the specified element.
7. Traverse the updated list using an iterator.
8. Display the remaining elements.

9. Stop.

PROGRAM

```
import java . util . * ;
public class Dlinkedlist {
public static void main ( String args [] ) {
LinkedList String ll = new LinkedList String();
ll . add ( "dinoy" );
ll . add ( "dhanya" );
ll . add ( "kuttan" );
ll . add ( "deepu" );
ll . add ( "sangi" );
System . out . println ( "List After Insertion : " );
Iterator String itr = ll . iterator ();
while ( itr . hasNext () ) {
System . out . println ( itr . next () );
}
ll . remove ( "dinoy" );
ll . remove ( 2 );
System . out . println ( "List After Removing : " );
Iterator String rti = ll . iterator ();
while ( rti . hasNext () ) {
System . out . println ( rti . next () );
}
}}
```

EXPECTED OUTPUT

List After Insertion

dinoy

dhanya

kuttan

deepu

sangi

List After Removing

dhanya

deepu

sangi

MODEL VIVA QUESTIONS

1. What is a linked list?
2. Difference between singly and doubly linked lists.
3. What is an iterator?
4. Which package contains `LinkedList`?
5. Difference between `ArrayList` and `LinkedList`.

RESULT

The Java program successfully demonstrated creation, deletion, and traversal of a doubly linked list.

EXP NO: 15**QUICK SORT****AIM**

To develop a Java program that implements the Quick Sort algorithm for sorting a list of strings in ascending order.

LEARNING OUTCOMES

Students will be able to:

- Understand the Divide and Conquer technique.
- Implement Quick Sort.
- Compare strings using Java methods.
- Perform recursive programming.
- Analyze sorting algorithms.

DESCRIPTION

Quick Sort is one of the fastest comparison-based sorting algorithms. It follows the Divide and Conquer strategy by selecting a pivot element, partitioning the array into smaller subarrays, and recursively sorting them.

The algorithm has an average time complexity of **$O(n \log n)$** and is widely used due to its efficiency in sorting large datasets.

ALGORITHM

1. Start.
2. Read the list of strings.
3. Select a pivot element.
4. Partition the list.
5. Recursively sort the left subarray.
6. Recursively sort the right subarray.
7. Display the sorted list.
8. Stop.

PROGRAM

```
import java . util . * ;
class quicksort {
String sortarr [] ;
int length ;
void input () {
int itr = 0 ;
Scanner sc = new Scanner ( System . in ) ;
System . out . println ( " \n Total Number Of String " ) ;
length = sc . nextInt () ;
this . sortarr = new String [ length ] ;
while ( length > itr ) {
System . out . print ( " \n " + ( itr + 1 ) + " th String : \n " ) ;
sortarr [ itr ] = sc . next () ;
itr ++ ;
}
sort ( 0 , length - 1 ) ;
}
void sort ( int l , int h ) {
int higher = h ;
int lower = l ; String pivot = sortarr [ lower + ( higher - lower ) / 2 ] ;
while ( lower <= higher ) {
while ( sortarr [ lower ]. compareToIgnoreCase ( pivot ) < 0 ) {
lower ++ ;
}
while ( sortarr [ higher ]. compareToIgnoreCase ( pivot ) > 0 ) {
higher -- ;
}
if ( lower <= higher ) {
swap ( higher , lower ) ;
lower ++ ;
higher -- ;
}
}
```

```
}  
if ( l < higher ){  
    sort ( l , higher );  
}  
if ( lower < h ) {  
    sort ( lower , h );  
}  
}  
  
void swap ( int higher , int lower ){  
    String temp = sortarr [ lower ];  
    sortarr [ lower ] = sortarr [ higher ];  
    sortarr [ higher ] = temp ;  
}  
  
void display (){  
    quicksort sort1 = new quicksort ();  
    for ( String c : sortarr ){  
        System . out . println ( " \n " + c );  
    }  
}  
  
public static void main ( String [] args ) {  
    Scanner s = new Scanner ( System . in );  
    quicksort st = new quicksort ();  
    int op = 0 ;  
    while ( true ){  
        System . out . println ( "Menu \n\n 1-input \n 2-sort and display \n 3-EXIT" );  
        op = s . nextInt ();  
        switch ( op ){  
            case 1 :  
                st . input ();  
                break ;  
            case 2 :  
                st . display ();  
                break ;  
            case 3 :  
                System . exit ( 0 );  
        }  
    }  
}
```

```
default:System . out . println ( "Error Select Correct Option" );  
}}}}
```

EXPECTED OUTPUT

Input

Kavitha

Arthi

Rohith

Sorted Output

Arthi

Kavitha

Rohith

MODEL VIVA QUESTIONS

1. What is Quick Sort?
2. What is a pivot element?
3. Explain Divide and Conquer.
4. What is partitioning?
5. Time complexity of Quick Sort.

RESULT

The Java program successfully sorted the given list of strings using the Quick Sort algorithm.

EXP NO: 16**BINARY SEARCH****AIM**

To develop a Java program that implements the Binary Search algorithm to search an element in a sorted array.

LEARNING OUTCOMES

Students will be able to:

- Understand searching techniques.
- Implement Binary Search.
- Apply iterative searching.
- Analyze algorithm efficiency.
- Work with sorted arrays.

DESCRIPTION

Binary Search is an efficient searching algorithm that works only on sorted arrays. The search space is repeatedly divided into two halves until the required element is found or the search interval becomes empty.

Its time complexity is **$O(\log n)$** , making it much faster than linear search for large datasets.

ALGORITHM

1. Start.
2. Read the sorted array.
3. Read the search element.
4. Find the middle element.
5. Compare the search element with the middle element.
6. If equal, display the position.
7. Otherwise search the left or right half.
8. Repeat until the element is found or the search interval is empty.
9. Stop.

PROGRAM

```
import java.util.Scanner;

class Main {
    int binarySearch(int array[], int element, int low, int high) {

        // Repeat until the pointers low and high meet each other
        while (low <= high) {

            // get index of mid element
            int mid = low + (high - low) / 2;

            // if element to be searched is the mid element
            if (array[mid] == element)
                return mid;

            // if element is less than mid element
            // search only the left side of mid
            if (array[mid] < element)
                low = mid + 1;

            // if element is greater than mid element
            // search only the right side of mid
            else
                high = mid - 1;
        }

        return -1;
    }

    public static void main(String args[]) {

        // create an object of Main class
        Main obj = new Main();

        // create a sorted array
        int[] array = { 3, 4, 5, 6, 7, 8, 9 };
```

```
int n = array.length;

// get input from user for element to be searched
Scanner input = new Scanner(System.in);

System.out.println("Enter element to be searched:");

// element to be searched
int element = input.nextInt();
input.close();

// call the binary search method
// pass arguments: array, element, index of first and last element
int result = obj.binarySearch(array, element, 0, n - 1);
if (result == -1)
    System.out.println("Not found");
else
    System.out.println("Element found at index " + result);
}
```

EXPECTED OUTPUT

Enter element to be searched:
6
Element found at index 3

MODEL VIVA QUESTIONS

1. What is Binary Search?
2. Why must the array be sorted?
3. What is the time complexity of Binary Search?
4. Difference between Linear Search and Binary Search.
5. What is the middle index?
6. Can Binary Search be applied to linked lists?
7. What happens if the element is not found?
8. What is iterative Binary Search?
9. What is recursive Binary Search?
10. Applications of Binary Search.

RESULT

The Java program was successfully implemented and executed. The Binary Search algorithm correctly located the required element in the sorted array.